Unità 1 · I database relazionali

Lezione 1 – Introduzione ai database

Dispensa per la classe 3 AFM · Informatica · Prof. Luigi Lomasto

IN QUESTA LEZIONE IMPAREREMO A…

- capire quali sono i **limiti degli archivi classici**;
- spiegare che cos'è un **DBMS**;
- riconoscere le **caratteristiche** e le **funzioni** di un DBMS.

1. Perché ci servono i database?

Qualsiasi programma, dalla app del meteo al gestionale di un'azienda, lavora con delle **informazioni**. Queste informazioni devono essere **salvate in modo permanente**, cioè restare memorizzate anche quando spegniamo il computer, per poterle usare di nuovo in seguito.

Salvare i dati "a mano" in **file di record** scritti con un linguaggio di programmazione è un lavoro lungo e faticoso. Inoltre i vecchi **archivi** su disco non sempre si adattano bene ai dati da gestire, e i problemi aumentano man mano che i dati crescono.

DA RICORDARE

Pensa alla differenza tra la **rubrica del tuo telefono** (poche centinaia di contatti) e la **contabilità di un'azienda** (migliaia di fatture, clienti, fornitori, pagamenti). Più i dati sono tanti e complessi, più serve uno strumento potente per gestirli.

In un'azienda le esigenze sono tante e diverse: gli **stipendi**, la **produzione**, la **contabilità**, il **magazzino**… Se ogni programma organizza i dati a modo suo, poi diventa difficile modificarlo o riutilizzarlo.

ESEMPIO

Pensa ai dati gestiti dall'**ufficio anagrafe** del Comune, dalla **contabilità** di una grande azienda, dalle **vendite** di un negozio online, dai **conti correnti** di una banca o dalle **prenotazioni** di un albergo o di una compagnia aerea.

Per gestire tutto questo in modo semplice, ordinato e flessibile si usano i **database**. I dati sono "il vero patrimonio" di ogni organizzazione, e ogni programma aziendale deve poter facilmente:

- **ricercare** e **recuperare** le informazioni secondo certi criteri (es. tutti i clienti di Napoli);
- **selezionarle** e raggrupparle (es. il totale delle vendite per mese);
- **aggiungerne** di nuove (es. un nuovo cliente);
- **modificarle** quando cambiano (es. un cliente cambia indirizzo);
- **cancellare** quelle che non servono più.

BASE DI DATI (DATABASE)

Una **base di dati** o **database** è una raccolta di dati progettata in modo da poter essere usata nel modo migliore da **applicazioni diverse** e da **utenti diversi**.

In questa unità studieremo la teoria comune a tutti i database e in particolare il **modello relazionale**, oggi il più usato in assoluto. Il suo vantaggio è che si basa su una teoria matematica solida e conosciuta in tutto il mondo.

DA RICORDARE

Che il database sia prodotto da **Oracle**, da **Microsoft** o da altri, la teoria alla base è la stessa: una volta imparata, sai usare qualsiasi database relazionale.

2. Sistema informativo e sistema informatico

Le informazioni di un'organizzazione (azienda, scuola, Comune…) sono gestite dal suo **sistema informativo**.

SISTEMA INFORMATIVO

È l'insieme organizzato di **strumenti automatici**, **procedure manuali**, **persone**, **risorse materiali** e **regole organizzative** che serve a gestire le informazioni importanti per un'organizzazione.

Quindi **ogni** informazione dell'azienda fa parte del sistema informativo, anche un foglio scritto a mano o una telefonata. Una parte di questo sistema è automatizzata: il **sistema informatico**.

SISTEMA INFORMATICO

Il **sistema informatico** (detto anche **EDP**, Electronic Data Processing) è la parte del sistema informativo che gestisce le informazioni in **modo automatico**, cioè sotto forma di **dati digitali**.

ESEMPIO

In un'azienda commerciale il **sistema informativo** comprende anche l'impiegato che risponde al telefono, gli ordini scritti su carta, le riunioni. Il **sistema informatico** è invece il software gestionale, il database delle fatture, i computer e la rete.

Il sistema informatico (SI) è formato da:

- i **supporti fisici** dove sono memorizzati i dati (dischi, server…);
- le **procedure** per cercare le informazioni, cioè le **applicazioni**;
- gli strumenti di **comunicazione** tra i terminali degli operatori (la rete).

In base alla loro natura distinguiamo due componenti:

- **componente software**: archivi e applicazioni;
- **componente hardware**: supporti fisici e strumentazione.

Noi ci occuperemo della **componente software**.

3. Archivi e applicazioni

ARCHIVIO E APPLICAZIONE INFORMATICA

L'**archivio** è l'insieme dei dati salvati su un **supporto di memorizzazione**.

L'**applicazione informatica** è il programma che usa i dati dell'archivio per svolgere una **funzione specifica** nell'organizzazione (es. fatturazione, paghe, magazzino).

L'organizzazione tradizionale (senza database)

Immaginiamo un'azienda che non usa un database. Ogni applicazione ha **i suoi dati** e la **sua struttura**, decisa da chi l'ha programmata: l'Applicazione 1 usa i Dati 1, l'Applicazione 2 usa i Dati 2, e così via. Ogni applicazione lavora **da sola**, in modo quasi del tutto indipendente dalle altre.

Le applicazioni sono quindi **isolate** anche se fanno parte dello **stesso sistema**. Questo crea problemi quando due applicazioni devono **condividere** gli stessi dati, come succede tra la **fatturazione** e la **contabilità**.

> **ESEMPIO**
>
> Due programmi usano lo stesso file. Uno dei due ha bisogno di **aggiungere un nuovo campo** (ad esempio l'email del cliente). Risultato: bisogna modificare anche la **descrizione dei dati** dentro l'altro programma, anche se a lui l'email non serve affatto!

Spesso modificare tutte le applicazioni insieme è troppo complicato, e si finisce per **duplicare i file**: gli stessi dati vengono copiati tante volte quante sono le applicazioni che li usano. Conseguenze: **spreco di memoria** e rischio di **errori** e **dati incoerenti**.

Inoltre si può **accedere ai dati** solo attraverso le applicazioni che li hanno creati. Se serve un'informazione nuova (un nuovo report, una nuova statistica) bisogna scrivere un nuovo programma, con grande spreco di **tempo** e **denaro**.

4. Dati, archivi e database

Per risolvere questi problemi serve un sistema capace di "contenere" **tutti i dati** di tutte le applicazioni, in **una sola copia**, unica e sempre aggiornata, disponibile per tutti i programmi. Questo sistema deve anche permettere l'**accesso simultaneo** (contemporaneo) di **più utenti**, anche con applicazioni scritte in linguaggi diversi.

Deve inoltre offrire un'**interfaccia** semplice, un **linguaggio specifico** per salvare, modificare e recuperare i dati, e deve permettere di cambiare la struttura dei dati **senza rovinare** i programmi che già funzionano.

Questo strumento è proprio il **database**: l'insieme degli archivi (**tabelle**) di tutte le applicazioni aziendali, che raccoglie tutti i dati, li organizza e li gestisce.

> **DA RICORDARE**
>
> La caratteristica principale di un database è **separare completamente i dati dalle applicazioni**.

Quando programmiamo, per salvare dei dati definiamo prima un **tipo di dato strutturato**, il **record** (ad esempio: nome, cognome, età). Senza accorgercene, abbiamo già separato i dati dalla loro **definizione**, cioè dalla descrizione di come sono fatti.

> **DA RICORDARE**
>
> Un passo fondamentale per definire i dati di un database è **descrivere come sono fatti i record**: quanti campi contengono, come si chiamano, di che tipo sono, più altre informazioni utili.

Se tutte le applicazioni devono usare gli stessi archivi, la **descrizione dei dati** deve stare **insieme ai dati stessi**, così qualunque programma può interpretarli correttamente.

ESEMPIO

Prova ad aprire un file di dati con il Blocco note: vedrai simboli incomprensibili. Senza la loro **definizione**, i dati da soli sono praticamente **inutili**.

DA RICORDARE

Nei database la **definizione dei dati** e i **dati stessi** sono salvati **nello stesso contenitore**: il database.

- Con gli **archivi tradizionali** la definizione dei dati sta **dentro l'applicazione**: Utente → Applicazione (+ definizione dati) → Archivio.
- Con un **database** la definizione dei dati è **indipendente** dall'applicazione: Utente → Applicazione → Database (+ definizione dati).

Serve però un "intermediario" tra le applicazioni e i dati, capace di leggere la definizione dei dati, trovare dove sono fisicamente e gestirli. Questo intermediario si chiama **DBMS**.

DBMS

Il **DBMS** (**Database Management System**) è il software che gestisce il database nel suo complesso. Si occupa di **tutto**: dei dati, della loro definizione e di come vengono fisicamente salvati.

ESEMPIO

Esempi di DBMS che probabilmente hai già sentito: **MySQL**, **Microsoft Access**, **Oracle**, **PostgreSQL**.

5. Perché separare dati e applicazioni: due esempi

Esempio 1 – Il magazzino

Un cliente vuole un programma per il suo **magazzino**. Chi registra il **carico** (merce in arrivo) usa un normale PC. Chi registra lo **scarico** (merce venduta) usa un **terminale portatile** che legge i codici a barre.

Il problema: il terminale è un piccolo computer, meno flessibile di un PC, e probabilmente usa un **linguaggio di programmazione diverso** (ad esempio Java o C#). Se i dati fossero "chiusi" dentro il programma del PC, il terminale non potrebbe usarli. Separare dati e applicazioni diventa quindi **indispensabile**.

Esempio 2 – Clienti e ordini

Un'impresa vende materiale elettrico e registra i dati in due archivi tradizionali: **clienti** e **ordini**.

clienti

cod_cliente	cognome	nome	indirizzo	città
010	Rossi	Pino	Via Milano, 15	Como
020	Verdi	Giuseppe	Via Italia, 1	Milano
030	Neri	Piero	Via Colombo, 3	Varese

ordini

cod_cliente	cod_articolo	descrizione	prezzo	quantità
010	M03	Lampadina LED 20 W	2,00	3

cod_cliente	cod_articolo	descrizione	prezzo	quantità
010	M12	Deviatore	3,00	1
010	M04	Antenna	25,00	3
020	M06	Trasformatore	60,00	2
020	M03	Lampadina LED 20 W	2,00	2
020	M12	Deviatore	3,00	1
030	M06	Trasformatore	60,00	2

Guardando la tabella ordini notiamo subito alcuni problemi:

- **Ridondanza dei dati**: la descrizione e il prezzo dell'articolo sono **ripetuti** in ogni ordine (la "Lampadina LED 20 W" compare due volte).
- **Incongruenza**: se il prezzo della lampadina aumenta, bisogna correggerlo in **tutte** le righe in cui compare.
- **Inconsistenza**: se ci dimentichiamo di correggerne una, avremo **due prezzi diversi** per la stessa lampadina… e non sapremo più quanto costa davvero!

A questi si aggiungono i problemi già visti per le applicazioni:

- **dipendenza logica**: se cambi la struttura di un record devi modificare **a cascata** tutti i programmi che usano quel file;
- **dipendenza fisica**: il modo in cui sono organizzati gli archivi limita le operazioni che il programmatore può fare, e nuove esigenze possono diventare irrealizzabili;
- i dati possono essere elaborati solo leggendo il file **record per record**.

> **DA RICORDARE**
>
> La teoria dei database propone un nuovo modo di organizzare gli archivi, con l'obiettivo di **superare tutti questi limiti**.

6. Le funzioni di un DBMS

Il DBMS si mette **in mezzo** tra i programmi e i file: salva i dati sui supporti fisici e li recupera quando un'applicazione li chiede. Poiché i file sono condivisi da più applicazioni, il DBMS garantisce anche la **mutua esclusione** (due programmi non possono modificare lo stesso dato nello stesso istante). In pratica, il DBMS è un **file system potenziato**.

Un DBMS deve:

- gestire **grandi quantità di dati**, più grandi della memoria centrale, salvandoli in **memoria secondaria**;
- garantire la **condivisione dei dati** tra applicazioni e utenti diversi;
- garantire la **persistenza dei dati**, che devono durare nel tempo anche dopo la chiusura dei programmi;
- garantire l'**affidabilità delle operazioni** (**transazioni**), che devono essere **atomiche** e non lasciare il database in uno stato sbagliato.

Il DBMS offre anche **interfacce grafiche** per l'amministrazione, incluse nel prodotto oppure realizzate da altri e spesso gratuite, come **HeidiSQL** per **MySQL**, il DBMS che useremo noi.

6.1 Gestione efficiente

Avere milioni di dati non serve a nulla se per trovarne uno bisogna aspettare un'eternità. Oggi vogliamo risposte in pochi secondi: il DBMS **non deve essere un collo di bottiglia** e deve permettere a più utenti di consultare i dati insieme.

Per questo usa strutture dati particolari (**alberi**, **tabelle hash**) e soprattutto gli **indici**.

> **INDICE**
>
> L'**indice** è una struttura che contiene la **posizione** dei record in base al valore di un **campo di ricerca** (la **chiave**). Funziona come l'indice di un libro: invece di sfogliare tutte le pagine, vai direttamente al capitolo che ti interessa (**accesso diretto**).

6.2 Condivisione e gestione della concorrenza

Se più utenti lavorano sugli stessi dati nello stesso momento, le loro operazioni potrebbero entrare in conflitto. Il DBMS deve **coordinare gli accessi** per evitare che qualcuno legga dati sbagliati o non aggiornati.

Lo fa con la **sincronizzazione** e con il **locking** (blocco): mentre un utente modifica un dato, quel dato viene "bloccato" per gli altri. Il DBMS può bloccare un'intera tabella, una pagina, una sola riga o perfino un solo campo (**granularità di locking**).

> **ESEMPIO**
>
> Due commessi di un negozio online vendono **l'ultimo paio di scarpe** nello stesso istante. Senza controllo della concorrenza il sistema potrebbe venderlo due volte! Il DBMS blocca il dato finché la prima vendita non è conclusa.

6.3 Persistenza e affidabilità: le transazioni

Il DBMS deve garantire l'**affidabilità dei dati** anche durante operazioni complesse e delicate, chiamate **transazioni**.

> **TRANSAZIONE**
>
> La **transazione** è un'unità logica di elaborazione formata da una serie di operazioni elementari (letture o scritture) sul database, che ne modificano lo stato.

Le transazioni devono rispettare le proprietà **ACID**:

- **Atomicity** (atomicità): la transazione si esegue **tutta** oppure **per niente**, come se fosse un'unica operazione indivisibile;
- **Consistency** (coerenza): alla fine il database deve trovarsi in uno stato **corretto**, come all'inizio;
- **Isolation** (isolamento): gli altri non devono vedere i risultati "a metà" di una transazione, ma solo quelli finali;
- **Durability** (persistenza): una volta conclusa, i risultati devono restare salvati **per sempre**.

Se anche una sola operazione fallisce, si attiva il **rollback**, che riporta il database com'era **prima** dell'inizio della transazione.

ESEMPIO

Un acquisto su un sito di e-commerce è una transazione fatta così:

1. il cliente mette i prodotti nel carrello e paga tramite la sua banca;
2. la banca addebita la spesa sul conto del cliente;
3. lo stesso importo viene comunicato come accredito per il conto del negozio;
4. la banca del negozio prende in carico l'accredito;
5. la banca del negozio registra l'importo e aggiorna il saldo;
6. il negozio riceve l'avviso del pagamento e spedisce la merce.

Se il sistema si blocca dopo il passo 2, il cliente ha pagato ma il negozio non ha ricevuto nulla: il cliente aspetterebbe la merce **invano**!

DA RICORDARE

Se la transazione non va a buon fine, tutte le operazioni devono essere **annullate**: il database torna allo stato iniziale, **come se nulla fosse successo**.

6.4 Sicurezza

Con tanti utenti aumenta il rischio che qualcuno, per sbaglio o con cattive intenzioni, veda dati che non dovrebbe. Il DBMS gestisce il **controllo degli accessi**: ogni utente o gruppo vede e modifica **solo** i dati per cui ha i permessi.

ESEMPIO

Nel registro elettronico lo studente vede solo i **propri voti**, il docente vede i voti delle **sue classi**, la segreteria vede i dati **anagrafici** di tutti. Stesso database, permessi diversi.

7. I DBMS aziendali

Ogni organizzazione (azienda, ufficio, ente, scuola) ha un sistema informatico che gestisce le sue informazioni sotto forma di dati.

Negli ultimi anni **social network**, **e-commerce**, **Internet** e **IoT** (Internet delle cose) hanno prodotto una quantità "spaventosa" di dati. Per questo oggi i database si dividono in due grandi famiglie:

- **database relazionali**, che useremo noi;
- **database non relazionali**, chiamati **NoSQL**.

I database relazionali usano un linguaggio standard per definire e interrogare i dati: l'**SQL** (**Structured Query Language**).

Ne esistono moltissimi, alcuni a pagamento (**commerciali**) e altri gratuiti (**open source**):

Principali DBMS relazionali

Commerciali	Commerciali	Commerciali	Open source
Oracle	IBM DB2	Informix	MySQL
MS SQL Server	SAP HANA	Ingres	PostgreSQL
MS Access	Sybase	Kinetica	Express-C

DA RICORDARE

MySQL è il database libero più diffuso sul Web, distribuito con licenza **open GPL**. Oggi è di proprietà di **Oracle**, che però l'ha mantenuto gratuito.

8. L'architettura ANSI/SPARC a tre livelli

ANSI/SPARC

È la sigla di **American National Standards Institute, Standards Planning And Requirements Committee**. Nel 1975 questo comitato propose un modello di architettura standard per i database, che ancora oggi è il punto di riferimento.

L'architettura ANSI/SPARC divide il database in **tre livelli**:

- **Schema esterno**: è la parte del database che interessa a un certo utente o a una certa applicazione. Sono "**viste**" parziali dei dati.
- **Schema logico**: è la descrizione dell'**intero** database secondo il modello usato dal DBMS (per noi, il modello relazionale a tabelle).
- **Schema interno** (o **fisico**): è il modo in cui lo schema logico viene salvato **fisicamente** su disco (file, indici…).

Dall'alto verso il basso: gli **utenti** usano le **viste** (schemi esterni) → tutte le viste si basano sullo **schema logico** → che è memorizzato nello **schema interno** → che contiene i **dati**.

L'utente quindi non vede mai tutto lo schema logico, ma solo la parte che serve alla sua applicazione: la **vista**.

ESEMPIO

Nel database di una scuola ci sono due tabelle: **aule** e **materie**.

aule

nome	edificio	piano
NB23	Centrale	Terra
AU12	Ala_est	Primo
XY12	Ala_est	Secondo
ZZ12	Ala_sud	Terzo

materie

corso	docente	aula
Informatica	Verdi	AU12
Educazione fisica	Rossi	XY12
Matematica	Bianchi	NB23
Lingua inglese	Gialli	XY12

La terza tabella **non è salvata** nel database: è una **vista** "ricavata" dalle altre due, che dice in quale edificio e piano si svolge ogni corso.

vista materie/aule

corso	aula	edificio	piano
Informatica	AU12	Ala_est	Primo
Educazione fisica	XY12	Ala_est	Secondo
Matematica	NB23	Centrale	Terra
Lingua inglese	XY12	Ala_est	Secondo

Un secondo esempio, tipico del mondo **AFM**: una tabella con i dati anagrafici di alcune ditte clienti e una con i dati delle città.

clienti

ragione sociale	indirizzo	città
Dadi e viti S.p.A.	Via Piave, 15	Bergamo
Tutto per il bricolage	V.le Arno, 155	Milano
Fai da te S.p.A.	Via Verdi, 12	Como
La casa del cacciavite	Via Rossi, 8	Brescia

residenze

città	sigla	cap
Brescia	BS	25100
Milano	MI	20100
Bergamo	BG	24100
Como	CO	22100

Unendo i dati delle due tabelle si ottiene una vista più completa, utile ad esempio per stampare gli indirizzi sulle **buste della corrispondenza** commerciale:

vista per la corrispondenza

ragione sociale	indirizzo	città	cap
Dadi e viti S.p.A.	Via Piave, 15	Bergamo	24100
Tutto per il bricolage	V.le Arno, 155	Milano	20100
Fai da te S.p.A.	Via Verdi, 12	Como	22100
La casa del cacciavite	Via Rossi, 8	Brescia	25100

I DBMS che studieremo rispettano l'architettura ANSI/SPARC. Impareremo a **progettare** un database fino a definire lo **schema logico** e poi a creare le tabelle con lo **schema relazionale**.

9. Ripasso: che cosa abbiamo imparato?

PAROLE CHIAVE

- **Database**: raccolta di dati usata da più applicazioni e più utenti.
- **DBMS**: software che gestisce il database (es. MySQL, Access).
- **Sistema informativo**: tutto ciò che gestisce le informazioni di un'organizzazione.
- **Sistema informatico**: la parte automatizzata, con dati digitali.
- **Ridondanza**: lo stesso dato ripetuto più volte.
- **Incongruenza / inconsistenza**: dati ripetuti che diventano diversi tra loro.
- **Transazione**: insieme di operazioni da fare tutte o nessuna (ACID, rollback).
- **Indice**: struttura per trovare velocemente i record.
- **Vista**: la porzione di database che vede un utente.
- **ANSI/SPARC**: architettura a tre livelli (esterno, logico, interno).

10. Ora prova tu a rispondere

1. Che cosa si intende per sistema informativo?
2. Che cosa si intende per sistema informatico? Che differenza c'è con il sistema informativo?
3. Quali sono i limiti degli archivi tradizionali?
4. Che cos'è una base di dati?
5. Che cosa si intende per DBMS?
6. Quali sono le funzioni di un DBMS?
7. Che cosa significa la sigla ACID? Spiega ogni proprietà con parole tue.
8. Com'è organizzata l'architettura ANSI/SPARC?
9. Che cosa si intende per ridondanza dei dati?
10. Quando si verifica l'incongruenza dei dati?
11. Quali sono le possibili cause che portano all'inconsistenza di un dato?

Esercizio di applicazione

Una piccola agenzia di viaggi registra le prenotazioni in un unico foglio con le colonne: nome cliente, telefono cliente, destinazione, prezzo del pacchetto, data partenza. Il signor Esposito ha prenotato 3 viaggi diversi.

a) Quali dati vengono ripetuti (ridondanza)?

b) Che cosa succede se il signor Esposito cambia numero di telefono e l'impiegato lo aggiorna solo in una riga?

c) Come potresti dividere i dati in due tabelle per evitare il problema?

d) Il pagamento online del viaggio è una transazione: elenca le operazioni che la compongono e spiega cosa deve fare il DBMS se il pagamento si interrompe a metà.